Surrogate Aerodynamic Modeling: Predicting Transonic Airfoil Coefficients via Machine Learning

Ashish Dasu
Khoury College of Computer Sciences, Northeastern University
dasu.a@northeastern.edu

March 2026

Abstract. High-fidelity computational fluid dynamics (CFD) simulation of transonic airfoil aerodynamics requires hours of compute per configuration, making it intractable for applications that demand thousands of evaluations such as trajectory optimization and real-time guidance, navigation, and control (GNC). This report documents a machine learning surrogate modeling project targeting the transonic regime (Mach 0.3–0.8), where shock wave formation and drag divergence create sharp nonlinearities that challenge classical interpolation. We propose training four regression models — polynomial regression, random forest, gradient boosting (XGBoost), and a PyTorch deep neural network — to simultaneously predict lift (C_l), drag (C_d), and pitching moment (C_m) coefficients from airfoil geometry and flight conditions. Models are evaluated on the publicly available TransonicSurrogate dataset [1]: 1,362 high-fidelity CFD simulations across eight airfoil geometries generated via the `rhoCentralFoam` compressible Euler solver. We detail the full implementation pipeline from EDA through hyperparameter tuning and physical consistency evaluation, with the goal of deploying a surrogate that achieves $R^2 \geq 0.99$ on held-out test data while reducing query latency from ∼7 minutes to under 1 millisecond.

1 Introduction

Aerodynamic coefficient prediction is central to every phase of flight vehicle design. The lift coefficient C_l, drag coefficient C_d, and pitching moment coefficient C_m determine the forces and moments acting on a vehicle as functions of Mach number, angle of attack (AoA), and geometry. They are required inputs for trajectory optimization, GNC controller design, structural load estimation, and thermal analysis.

The standard approach is high-fidelity CFD: solve the governing Navier-Stokes or Euler equations on a fine computational mesh, resolving the pressure distribution and integrating surface forces. This yields accurate results but at prohibitive computational cost — a single steady-state simulation for a 2D airfoil in the transonic regime requires tens of minutes to hours on modern hardware clusters. For design optimization, Monte Carlo uncertainty quantification, or real-time onboard GNC, the requirement for thousands to millions of function evaluations renders direct CFD infeasible.

Machine learning surrogates offer a practical solution: given a library of pre-computed CFD results, train a regression model that maps flight conditions and geometry to aerodynamic coefficients. At inference, the model replaces the CFD call entirely, reducing query time from minutes to microseconds. This approach is already used in industry for preliminary design and real-time simulation [2]. The scientific challenge is achieving accuracy and generalization across the input space, particularly in flow regimes where the underlying physics is strongly nonlinear.

1.1 The Transonic Regime

The transonic regime (approximately Mach 0.3–0.8) is where this modeling challenge is most acute. As freestream velocity increases toward the speed of sound, local flow accelerations over the upper airfoil surface cause patches of supersonic flow to appear even when the freestream is still subsonic. These supersonic patches terminate in shock waves that interact with the boundary layer, causing *drag divergence*: a sharp, near-discontinuous rise in C_d

that simple smooth regressors cannot capture.

From an operational standpoint, the transonic regime governs several critical phases of modern launch vehicle flight: stage separation, grid fin actuation during first-stage reentry, and the subsonic-to-transonic transition of hypersonic reentry vehicles. Accurate, fast aerodynamic prediction in this regime is therefore of direct engineering relevance.

1.2 Contributions

This project makes the following concrete contributions:

1. A systematic comparison of four regression architectures on a real transonic CFD dataset, spanning model families from linear to ensemble to deep learning.
2. A complete, reproducible implementation pipeline from raw CSV to trained models, evaluation figures, and latency benchmarks.
3. Physical plausibility evaluation via drag polar comparison — beyond scalar metrics, we assess whether predicted C_l–C_d relationships are physically coherent.
4. Quantified inference speedup over direct CFD, demonstrating practical applicability for GNC and trajectory optimization.

2 Background and Related Work

2.1 Aerodynamic Coefficient Definitions

The three aerodynamic coefficients of interest are defined as:

$$C_l = \frac{L}{\frac{1}{2}\rho V^2 A}, \quad C_d = \frac{D}{\frac{1}{2}\rho V^2 A}, \quad C_m = \frac{M}{\frac{1}{2}\rho V^2 A c} \tag{1}$$

where L, D, M are lift force, drag force, and pitching moment; ρ is freestream density; V is freestream velocity; A is reference area; and c is reference chord length. These non-dimensional forms allow coefficients to be tabulated as functions of Mach number M_∞ and angle of attack α, independent of vehicle scale.

Drag Divergence

As M_∞ approaches the *critical Mach number* M_{cr} (typically 0.7–0.85 depending on airfoil camber and thickness), local supersonic zones form on the upper surface. These terminate in normal shocks that cause boundary layer separation, producing a sharp rise in C_d. The critical Mach number and the steepness of the divergence depend strongly on airfoil geometry — thinner, less cambered profiles diverge at higher Mach numbers, which is why transonic aircraft use supercritical airfoil designs (e.g. RAE2822).

2.2 ML Surrogate Models in Aerodynamics

Data-driven surrogate modeling for aerodynamics has an extensive literature. Bouhlel et al. [2] demonstrated gradient-enhanced neural networks (GENN) for subsonic and transonic airfoil C_d prediction, using mode shape parameterization to represent geometry in 16 features. Elrefaie et al. [1] extended this to the full transonic envelope with ensemble and deep learning methods, reporting $R^2 = 99.6\%$ with their best model on eight airfoil geometries. Sekar et al. applied convolutional networks to predict full-field pressure distributions from airfoil geometry, from which lift and drag are integrated. More recently, neural fields and operator-based approaches [5] have targeted generalization across unseen geometries without requiring geometric parameterization.

Our work most directly mirrors [1], using the same dataset but providing a more systematic regression comparison, including a polynomial baseline absent from their study, and adding physical consistency evaluation via drag polars.

3 Dataset

3.1 TransonicSurrogate

We use the TransonicSurrogate dataset [1], publicly available at `https://github.com/Mohamedelrefaie/TransonicSurrogate`. The dataset consists of **1,362 high-fidelity CFD simulations** generated in OpenFOAM using the `rhoCentralFoam` density-based compressible flow solver, which discretizes the compressible

Euler equations using the Kurganov-Tadmor central-upwind scheme. Notably, this is an *inviscid* solver — viscous and turbulence effects are not modeled. This is appropriate for transonic aerodynamic coefficient prediction where pressure forces dominate, and significantly reduces per-simulation cost (approximately 7 minutes on 128 CPU cores and 4 NVIDIA A100 GPUs).

3.2 Airfoil Geometries

Eight airfoil profiles are included, spanning symmetric and cambered families:

- **RAE2822, RAE5212:** British transonic research airfoils widely used as CFD validation benchmarks. RAE2822 is the canonical transonic test case in the aerodynamics literature.
- **NACA0012:** Symmetric (0% camber, 12% thickness), the most widely studied airfoil in the world.
- **NACA2412, NACA4412:** Cambered NACA 4-digit series (2% and 4% max camber, 12% thickness).
- **NACA23012, NACA24112, NACA25112:** NACA 5-digit series, with forward-loaded camber lines.

3.3 Operating Conditions

Simulations cover:

- Freestream Mach number: $M_\infty \in [0.3, 0.8]$
- Angle of attack: $\alpha \in [-4°, +8°]$

Together with the 8 airfoil geometries, this results in a dataset that densely covers the transonic envelope, including the drag divergence region for each profile.

3.4 Feature Representation

Each sample is represented by a numeric feature vector combining M_∞, α, and geometric parameters characterizing the airfoil profile (thickness, camber, and related shape descriptors). The authors deliberately use numeric parameterization rather than one-hot encoding the airfoil identity [1]: this forces the model to learn how geometry affects aerodynamics continuously rather than memorizing per-airfoil lookup tables, which is necessary for any generalization to unseen shapes.

3.5 Dataset Statistics

Property	Value
Total samples	1,362
Airfoil geometries	8
Mach range	0.3 – 0.8
AoA range	−4 – +8
CFD solver	rhoCentralFoam
Equations	Compressible Euler
Compute/sim	∼7 min (128 cores + 4×A100)
Output targets	3 (C_l, C_d, C_m)
Train / Val / Test	60 / 20 / 20

3.6 Preprocessing

All input features are standardized to zero mean and unit variance using Z-score normalization:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j} \tag{2}$$

where μ_j and σ_j are computed on the *training set only*. The same transformation is applied to validation and test sets using training statistics. Output coefficients are normalized per-target independently to prevent the loss from being dominated by the target with the largest absolute scale. Scalers are saved and applied in reverse at evaluation time so all reported metrics are in original physical units.

4 Problem Formulation

Let $\mathbf{x} = [M_\infty, \alpha, g_1, \ldots, g_k]^\top$ denote the input feature vector combining flight conditions and k airfoil geometric parameters. Let $\mathbf{y} = [C_l, C_d, C_m]^\top$ denote the vector of aerodynamic coefficient targets. We seek a function $f_\theta : \mathbb{R}^{k+2} \to \mathbb{R}^3$ that minimizes the expected prediction error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\mathbf{x},\mathbf{y})}\left[\sum_{i=1}^{3} \left(f_\theta(\mathbf{x})_i - y_i\right)^2\right] \tag{3}$$

This is a **multi-output regression** problem. We compare four function families f_θ with increasing representational capacity, from polynomial to deep neural network. The key scientific question is: *how much model complexity is required to capture the nonlinear transonic aerodynamic response, and does additional complexity beyond ensemble methods provide measurable gains on a dataset of this scale?*

5 Methodology

5.1 Model 1: Polynomial Regression

As a smooth analytic baseline, we apply degree-d polynomial basis function expansion to all inputs, then fit a ridge-regularized linear model. For inputs $\mathbf{x} \in \mathbb{R}^p$, the degree-$d$ expansion maps to a feature space of dimension $\binom{p+d}{d}$, exponentially larger than p. Ridge regularization (ℓ_2 penalty with coefficient λ) is applied to prevent coefficient explosion in the expanded space:

$$\hat{\theta} = \arg\min_{\theta} \|\Phi\theta - Y\|^2 + \lambda\|\theta\|^2 \tag{4}$$

where Φ is the polynomial feature matrix. Degree $d \in \{2, 3\}$ is selected via 5-fold cross-validation on the training set.

Hypothesis: Polynomial regression will fail to capture drag divergence accurately. The sharp nonlinearity in C_d near $M_\infty = 0.8$ cannot be represented by a globally smooth low-degree polynomial, so this model will show systematic bias in the high-Mach regime while performing reasonably well at lower Mach numbers where the response is smoother.

5.2 Model 2: Random Forest Regressor

Random Forest [3] builds an ensemble of N decision trees on bootstrap samples of the training data, with each split considering a random subset of features. The final prediction is the mean across all trees. Multi-output regression is natively supported by applying the same splitting criterion jointly across all targets.

The ensemble mechanism reduces variance without increasing bias, and decision boundaries allow the model to capture the discontinuous behavior near drag divergence without feature engineering. We tune:

- Number of trees: $N \in \{50, 100, 200\}$
- Maximum depth: $d \in \{5, 10, \infty\}$
- Minimum samples per split: $\{2, 5\}$

via 5-fold CV maximizing mean R^2.

Hypothesis: Random Forest will substantially outperform polynomial regression near drag divergence due to its ability to partition the Mach-AoA space with axis-aligned boundaries. However, it may underperform the DNN on extrapolation to airfoil shapes far from the training distribution.

5.3 Model 3: Gradient Boosting (XGBoost)

XGBoost [4] constructs an additive ensemble of shallow trees sequentially: each tree fits the residual of the previous ensemble, weighted by the negative gradient of the loss. Unlike Random Forest, which averages independent trees, XGBoost reduces bias directly by targeting the error of the current model. Since XGBoost does not natively support multi-output regression, we train one model per coefficient (C_l, C_d, C_m) independently. Hyperparameters:

- Learning rate: $\eta \in \{0.01, 0.05, 0.1\}$
- Maximum depth: $d \in \{3, 5, 7\}$
- Number of estimators: $N \in \{100, 300, 500\}$
- Subsample ratio: $s \in \{0.8, 1.0\}$

tuned via 5-fold CV per target.

Hypothesis: XGBoost will be the strongest non-DNN model. On tabular data of this dimensionality and scale, gradient boosting typically outperforms Random Forest by better targeting hard-to-fit regions. The separate-model-per- output training may be a limitation relative to the joint DNN, since it cannot exploit correlations between C_l, C_d, and C_m.

5.4 Model 4: Deep Neural Network (PyTorch)

We implement a fully-connected MLP with a shared trunk and three separate linear output heads:

$$f_\theta(\mathbf{x}) = \begin{bmatrix} W_{C_l}\phi_\theta(\mathbf{x}) \\ W_{C_d}\phi_\theta(\mathbf{x}) \\ W_{C_m}\phi_\theta(\mathbf{x}) \end{bmatrix} \tag{5}$$

where ϕ_θ is the shared trunk mapping inputs to a learned representation, and W_{C_l}, W_{C_d}, W_{C_m} are separate linear readout heads. The trunk consists of fully-connected layers with ReLU activations, batch normalization, and dropout regularization. All three heads are trained jointly minimizing sum MSE:

$$\mathcal{L} = \mathrm{MSE}(C_l) + \mathrm{MSE}(C_d) + \mathrm{MSE}(C_m) \tag{6}$$

Architecture hyperparameters are searched over:

- Hidden layer configurations: $[128, 128]$, $[256, 128]$, $[256, 256]$, $[256, 128, 64]$, $[512, 256, 128]$
- Dropout probability: $p \in \{0.1, 0.3\}$
- Learning rate: $\eta \in \{10^{-3}, 5 \times 10^{-4}\}$

Training uses Adam with a ReduceLROnPlateau scheduler (factor 0.5, patience 10 epochs) and early stopping (patience 30 epochs on validation loss).

Hypothesis: The joint shared-trunk training will give the DNN an advantage over XGBoost on C_m prediction specifically, since the trunk learns a representation informed by all three outputs simultaneously. The main risk is overfitting given 817 training samples — the dropout and early stopping are critical. We expect the DNN to match but not dramatically exceed XGBoost performance overall.

5.5 Evaluation Protocol

All hyperparameter selection is performed using 5-fold CV on the training set. The held-out test set (20%, 273 samples) is evaluated exactly once per model after all hyperparameters are fixed. We report:

1. R^2 and RMSE per output coefficient (C_l, C_d, C_m) on the test set, in original physical units after inverse-scaling.
2. **Drag polar comparison**: predicted C_l vs. C_d curves plotted against CFD ground truth, filtered by airfoil and binned by Mach number. This evaluates physical coherence beyond scalar metrics — a model can achieve high per-coefficient R^2 while producing physically nonsensical relationships between coefficients.
3. **Residual analysis by Mach regime**: error is reported separately for sub-divergence ($M < 0.7$), divergence ($M \in [0.7, 0.85]$), and post-divergence ($M > 0.85$) regimes, to identify where each model fails.
4. **Inference latency**: single-sample prediction time measured over 1,000 runs, compared to the ∼7 minute CFD baseline.

6 Implementation

6.1 Repository Structure

```
cs6140-aero-surrogate/
  data/
    Input_Data.csv          # raw dataset from GitHub
  src/
    data.py                 # loading, splitting, scaling
    models.py               # sklearn model configs
    dnn.py                  # PyTorch architecture
    train.py                # training loops
    evaluate.py             # metrics, figures
  notebooks/
    01_eda.ipynb            # exploratory analysis
    02_polynomial.ipynb     # baseline
    03_random_forest.ipynb
    04_xgboost.ipynb
    05_dnn.ipynb
    06_evaluation.ipynb     # all figures, tables
  results/
    figures/
    tables/
  requirements.txt
  README.md
```

6.2 Data Pipeline

```python
# src/data.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

TARGET_COLS  = ['Cl', 'Cd', 'Cm']

def load_dataset(path='data/Input_Data.csv'):
    """Load CSV, separate features from targets."""
    df = pd.read_csv(path)
    feat_cols = [c for c in df.columns
                 if c not in TARGET_COLS]
    X = df[feat_cols].values.astype(np.float32)
    y = df[TARGET_COLS].values.astype(np.float32)
    return X, y, feat_cols

def make_splits(X, y, seed=42):
    """60 / 20 / 20 split matching original paper."""
    X_tmp, X_test, y_tmp, y_test = train_test_split(
```

```python
        X, y, test_size=0.20, random_state=seed)
    # 0.25 of remaining = 0.20 of total
    X_train, X_val, y_train, y_val = train_test_split(
        X_tmp, y_tmp, test_size=0.25, random_state=seed)
    return X_train, X_val, X_test, y_train, y_val, y_test

def fit_scalers(X_train, y_train):
    """Fit scalers on training data ONLY."""
    Xs = StandardScaler().fit(X_train)
    ys = StandardScaler().fit(y_train)
    return Xs, ys

def apply_scalers(Xs, ys, X_train, X_val, X_test,
                  y_train, y_val, y_test):
    return (Xs.transform(X_train),
            Xs.transform(X_val),
            Xs.transform(X_test),
            ys.transform(y_train),
            ys.transform(y_val),
            ys.transform(y_test))
```

6.3 DNN Architecture

```python
# src/dnn.py
import torch
import torch.nn as nn

class AeroSurrogate(nn.Module):
    """
    Multi-output MLP: shared trunk + 3 output heads.
    Joint training allows shared representation across
    Cl, Cd, Cm, exploiting inter-coefficient correlations.
    """
    def __init__(self, input_dim, hidden_dims,
                 n_outputs=3, dropout_p=0.1):
        super().__init__()
        layers, prev = [], input_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev, h),
                       nn.BatchNorm1d(h),
                       nn.ReLU(),
                       nn.Dropout(dropout_p)]
            prev = h
        self.trunk = nn.Sequential(*layers)
        # Separate heads per coefficient
        self.heads = nn.ModuleList(
            [nn.Linear(prev, 1) for _ in range(n_outputs)])

    def forward(self, x):
        z = self.trunk(x)
        return torch.cat([h(z) for h in self.heads], dim=1)
```

6.4 Training Loop with Early Stopping

```python
# src/train.py
import torch, copy
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

def train_dnn(model, X_tr, y_tr, X_val, y_val,
              lr=1e-3, epochs=500, batch=64,
              patience=30, device='cpu'):
    device = torch.device(device)
    model  = model.to(device)
    opt    = torch.optim.Adam(model.parameters(), lr=lr)
    sched  = torch.optim.lr_scheduler\
                .ReduceLROnPlateau(opt, patience=10,
                                   factor=0.5)

    loss_fn = nn.MSELoss()
    loader  = DataLoader(
        TensorDataset(torch.FloatTensor(X_tr),
                      torch.FloatTensor(y_tr)),
        batch_size=batch, shuffle=True)

    best_val, best_wts, wait = float('inf'), None, 0
    history = {'train': [], 'val': []}

    for ep in range(epochs):
        # ---- train ----
        model.train()
        tloss = 0.0
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            opt.step()
            tloss += loss.item() * len(xb)
        tloss /= len(X_tr)
```

```python

        # ---- validate ----
        model.eval()
        with torch.no_grad():
            Xv = torch.FloatTensor(X_val).to(device)
            yv = torch.FloatTensor(y_val).to(device)
            vloss = loss_fn(model(Xv), yv).item()

        sched.step(vloss)
        history['train'].append(tloss)
        history['val'].append(vloss)

        if vloss < best_val:
            best_val  = vloss
            best_wts  = copy.deepcopy(model.state_dict())
            wait      = 0
        else:
            wait += 1
            if wait >= patience:
                print(f'Early stop @ epoch {ep+1}')
                break

    model.load_state_dict(best_wts)
    return model, history
```

6.5 Evaluation and Figure Generation

```python
# src/evaluate.py
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score, mean_squared_error

def metrics_table(models, X_test, y_test, y_scaler,
                  names=['Cl','Cd','Cm']):
    """
    Return DataFrame: model x target -> R2, RMSE
    in original (inverse-scaled) units.
    """
    import pandas as pd
    rows = []
    y_true = y_scaler.inverse_transform(y_test)
    for mname, y_pred_norm in models.items():
        y_pred = y_scaler.inverse_transform(y_pred_norm)
        for i, t in enumerate(names):
            rows.append({
                'Model':  mname,
                'Target': t,
                'R2':   r2_score(y_true[:,i], y_pred[:,i]),
                'RMSE': mean_squared_error(y_true[:,i], y_pred[:,i], squared=False)})
    return pd.DataFrame(rows)

def plot_residuals_by_mach(y_true, y_pred, mach_vals,
                           model_name, save_dir):
    """
    Bin test set into 3 Mach regimes, plot RMSE per bin.
    Reveals where each model fails: pre/at/post divergence.
    """
    bins  = [(0.3, 0.7, 'Sub-divergence'),
             (0.7, 0.82,'Divergence'),
             (0.82,0.80,'Post-divergence')]
    fig, axes = plt.subplots(1, 3, figsize=(14,4),
                             sharey=False)
    for ax, (lo, hi, label) in zip(axes, bins):
        mask = (mach_vals >= lo) & (mach_vals < hi)
        if mask.sum() == 0:
            continue
        rmse = [mean_squared_error(y_true[mask,i], y_pred[mask,i],
                squared=False)
                for i in range(3)]
        ax.bar(['Cl','Cd','Cm'], rmse,
               color=['#1f77b4','#d62728','#2ca02c'])
        ax.set_title(f'{label}\n(n={mask.sum()})')
        ax.set_ylabel('RMSE')
    fig.suptitle(f'Residuals by Mach Regime  {model_name}',
                 fontweight='bold')
    plt.tight_layout()
    plt.savefig(f'{save_dir}/residuals_{model_name}.png',
                dpi=150)

def plot_drag_polar(y_true, preds_dict, airfoil_ids,
```

```python
                    mach_vals, save_dir):
    """
    For each unique Mach bin, plot Cl vs Cd curves.
    Compare CFD ground truth vs all models.
    """
    mach_bins = [0.5, 0.6, 0.7, 0.75, 0.8]
    tol = 0.02
    fig, axes = plt.subplots(1, len(mach_bins),
                             figsize=(4*len(mach_bins), 4))
    for ax, m in zip(axes, mach_bins):
        mask = np.abs(mach_vals - m) < tol
        if mask.sum() < 3:
            continue
        idx = np.argsort(y_true[mask, 0])  # sort by Cl
        ax.plot(y_true[mask,1][idx], y_true[mask,0][idx],
                'k-', lw=2, label='CFD', zorder=5)
        colors = ['#1f77b4','#d62728','#2ca02c','#ff7f0e']
        for (name, yp), c in zip(preds_dict.items(), colors):
            ax.plot(yp[mask,1][idx], yp[mask,0][idx],
                    '--', color=c, lw=1.5, label=name)
        ax.set_title(f'M = {m}')
        ax.set_xlabel('$C_d$')
        ax.set_ylabel('$C_l$')
    axes[0].legend(fontsize=7)
    plt.suptitle('Drag Polar Comparison', fontweight='bold')
    plt.tight_layout()
    plt.savefig(f'{save_dir}/drag_polars.png', dpi=150)
```

7 Expected Results and Hypotheses

Based on the dataset scale, the known physics of transonic aerodynamics, and published results from the original paper [1], we state the following testable hypotheses:

H1 — Polynomial failure at drag divergence. Polynomial regression will achieve $R^2 \geq 0.90$ on C_l (which varies smoothly with AoA at fixed Mach) but will show $R^2 < 0.80$ on C_d in the Mach 0.7–0.85 regime, where drag divergence creates a sharp nonlinearity. This should be visible as large systematic residuals in that Mach bin.

H2 — Ensemble models capture the discontinuity. Random Forest and XGBoost will both achieve $R^2 > 0.97$ on all three targets across the full test set, with XGBoost outperforming RF on C_d due to its residual-targeting approach being better suited to fitting sharp transitions.

H3 — DNN performance depends on regularization. With only 817 training samples, an unregularized DNN will overfit. With dropout $p \geq 0.1$ and early stopping, we expect the DNN to match XGBoost within measurement noise on C_l and C_d, and potentially outperform on C_m due to joint training capturing inter-coefficient structure.

H4 — Inference latency reduces by at least $10^4\times$. All models should achieve single-sample inference well under 1 ms, representing a speedup of $\geq 420,000\times$ over the 7-minute CFD baseline. This directly demonstrates practical utility for GNC and trajectory optimization.

H5 — Drag polar coherence degrades for simple models. Even if scalar R^2 values are similar across models, drag polar plots will reveal that polynomial regression and possibly RF produce physically incoherent C_l-C_d relationships at high Mach, while the DNN and XGBoost produce curves closer to the CFD ground truth.

8 Limitations and Scope

Dataset scale. 1,362 samples is small compared to typical ML benchmarks. Results may not generalize to the full space of transonic airfoil geometries beyond the 8 profiles included. The practical implication is that surrogate accuracy should be validated before deployment on geometries outside the training distribution.

Inviscid physics. The CFD data was generated with the Euler equations (inviscid), which do not model viscous boundary layer effects or turbulence. Real aerodynamic coefficients, especially C_d at low

Mach or near stall, include viscous contributions that this data cannot capture. The surrogate is valid only in the regime where pressure forces dominate.

2D airfoils only. All simulations are for 2D cross-sectional profiles. 3D wing effects (tip vortices, span loading, sweep effects) are not captured. Extending to 3D would require different datasets and significantly higher-dimensional feature representations.

Fixed geometry parameterization. The dataset contains only 8 airfoil profiles. While numeric parameterization enables some interpolation between geometries, true generalization to arbitrary unseen airfoil shapes is not achievable without a much larger and more diverse training set.

9 Conclusion

This report describes a rigorous machine learning surrogate modeling study targeting transonic airfoil aerodynamics — a problem of direct relevance to modern launch vehicle GNC and design. We have defined a complete research plan covering dataset analysis, four regression models across model complexity classes, a formal evaluation protocol including physical consistency checks, and a full reproducible implementation pipeline.

The central technical contribution is a systematic regression comparison on a real transonic CFD dataset, with explicit hypotheses about where each model family should succeed or fail. Beyond scalar metrics, the drag polar analysis and Mach-binned residual plots are designed to expose physically meaningful failure modes that R^2 alone cannot detect. All code, data, and results will be made publicly available upon project completion.

References

[1] M. Elrefaie, T. Ayman, M. Elrefaie, E. Sayed, M. Ayyad, and M. M. AbdelRahman, "Surrogate Modeling of the Aerodynamic Performance for Airfoils in Transonic Regime," *AIAA SciTech Forum*, 2024. https://github.com/Mohamedelrefaie/TransonicSurrogate

[2] M. A. Bouhlel, S. He, and J. R. R. A. Martins, "Scalable Gradient-Enhanced Artificial Neural Networks for Airfoil Shape Design in the Subsonic and Transonic Regimes," *Structural and Multidisciplinary Optimization*, vol. 61, 2020.

[3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *ACM SIGKDD*, 2016.

[5] G. Catalani et al., "Towards Scalable Surrogate Models Based on Neural Fields for Large Scale Aerodynamic Simulations," *arXiv preprint*, 2025.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.